A Hybrid Post-Quantum Key Encapsulation Mechanism: Composing RSA-2048 with Furui-Sieved Primes and ML-KEM-768

Ryoji Furui

Ryoji.info

May 2026

Abstract

We present a hybrid key-encapsulation mechanism (KEM) that composes RSA-2048 — instantiated with primes generated by the Furui 6n±1 sieve — with the NIST-standardized lattice-based ML-KEM-768 (FIPS 203). The shared secret is the output of HKDF-SHA-256 applied to the concatenation of both component secrets together with a transcript of the public keys and ciphertexts. Under the dual-PRF property of the combiner the hybrid is IND-CCA secure as long as either component remains secure: classical adversaries face the factoring barrier of RSA, while a future cryptographically relevant quantum computer (CRQC) running Shor's algorithm against RSA leaves the security of the hybrid to ML-KEM-768. The Furui sieve plays no security role; its contribution is faster RSA prime generation on Apple Silicon hardware during the transition window. We give a complete construction; a formal IND-CCA reduction with advantage bounds (Appendix A); rejection-sampling and empirical validation that the sieve produces primes whose distribution is indistinguishable from standard generators (Appendix B); a reproducible benchmark methodology (Appendix C); side-channel mitigations specific to the sieve and GPU code paths; key-rotation guidance; and parameter sizes for TLS 1.3 hybrid key exchange and HPKE.

Addendum on authorship and review status.

The 6n±1 prime formulation discussed in §2.2 and Appendix B is the author's original work, presented in the cited paper [3]. The remainder of this document — the hybrid construction in §3, the security analysis in §5, the implementation notes in §6, and the appendices — was drafted by Anthropic's Claude (commercial tier) at the author's direction, with reviewer comments from Microsoft Copilot incorporated during a subsequent revision pass. The author did not bring formal training in lattice cryptography or protocol-level security analysis to the work and has not obtained independent expert review. Readers should treat the construction and the security claims as a starting point for discussion rather than as peer-reviewed cryptographic guidance, and should obtain formal review before relying on the design in deployment.

1. Introduction

Shor's polynomial-time quantum algorithm for factoring and discrete logarithms [1] ends the era of public-key cryptography rooted in those problems. RSA, DSA, ECDSA, and classical Diffie–Hellman all fall to a sufficiently large fault-tolerant quantum computer. Public estimates put a CRQC capable of breaking RSA-2048 within the 2030s [2], with considerable uncertainty in either direction.

Three observations motivate hybrid post-quantum deployments today. First, “harvest now, decrypt later” attacks make traffic intercepted today vulnerable when quantum decryption arrives; post-quantum

confidentiality is therefore needed before the threat materializes. Second, lattice-based schemes are roughly a decade old as deployable primitives, so conservative designs hedge them with battle-tested classical constructions. Third, regulatory frameworks including the U.S. CNSA 2.0 [11] and the German BSI TR-02102 [12] explicitly endorse hybrid modes during the transition.

We construct a hybrid KEM that combines RSA-2048 (with Furui-sieved primes [3]) and ML-KEM-768 (FIPS 203, [4]). The output of each component is a 32-byte shared secret; the hybrid combines them with HKDF-SHA-256 to produce a single 32-byte shared secret robust to the failure of either component. The Furui sieve is a prime-generation primitive, not a cryptographic primitive: it accelerates RSA key generation on Apple Silicon (M1–M5) but is mathematically transparent, since the resulting primes are statistically identical to those produced by any other sound generator. We include it explicitly because efficient classical key generation matters during the transition, when systems will keep generating new RSA keys for years.

Contributions:

- A concrete construction and pseudocode for the hybrid KEM (§3), including explicit failure-handling and KDF-salt versioning.
- A formal IND-CCA security theorem with advantage bounds and a reduction to the RSA assumption, M-LWE, and HKDF's dual-PRF property (§5, Appendix A).
- A rejection-sampling argument and empirical Dirichlet-uniformity tests demonstrating that Furui-sieved primes are statistically indistinguishable from primes produced by standard generators (§4.2, Appendix B).
- Apple Silicon implementation notes with concrete side-channel mitigations for the sieve and GPU Miller–Rabin code paths (§6.1, §5.4).
- Parameter sizes, wire format, downgrade-resistance, and HPKE/TLS 1.3 integration notes (§6.2).
- Recommended key-rotation policy for the transition window (§7.5) and a reproducible benchmark methodology (Appendix C).

2. Background

2.1 RSA-KEM and the quantum threat

RSA-KEM [9] derives a shared secret from a random integer encrypted under the receiver's RSA public key. Given modulus $N = pq$ with public exponent e, the sender chooses $r \leftarrow \mathbb{Z}_N$ uniformly at random, computes $c = r^e \bmod N$, and outputs $K^{RSA} = \mathrm{KDF}(r)$. The receiver inverts with $K^{RSA} = \mathrm{KDF}(c^d \bmod N)$ using the private exponent d. Security of RSA-KEM reduces to the RSA assumption (and ultimately to the hardness of factoring N) in the random oracle model for the KDF.

Shor's algorithm factors $N = pq$ in time polynomial in $\log N$ on a quantum computer using $O(\log N)$ logical qubits and standard quantum-circuit optimizations. Once a CRQC exists with sufficient logical qubits and circuit fidelity, the RSA assumption fails completely — independent of how p and q were generated, how large they are within the polynomial regime, and what padding is used.

2.2 Furui's 6n±1 prime sieve

Furui [3] partitions the natural numbers into six columns indexed by $(2(3n-2), 3(2n-1), 2(3n-1), 6n-1, 2\cdot 3n, 6n+1)$. The columns $2(3n-2)$, $3(2n-1)$, $2(3n-1)$, and $2\cdot 3n$ always contain the factor 2 or 3, so every

prime greater than 3 lies in column 5 or column 7 — equivalently, every prime $p > 3$ satisfies $p \equiv \pm 1 \pmod 6$.

Furthermore, every composite of the form $6n\pm1$ must factor into two such forms: the smallest prime factor of any composite $c = 6n\pm1$ is at least 5 and is itself of the form $6m\pm1$; the cofactor is composed of primes ≥ 5, which (closed under multiplication mod 6) again yields a $6m\pm1$ form. Hence every composite $c = 6n\pm1$ admits a factorization $c = (6a\pm1)(6b\pm1)$ for some $a, b \in \mathbb{N}$. This yields a wheel sieve: enumerate candidates of the form $6n\pm1$ in $[L, U]$ and exclude every product $(6a\pm1)(6b\pm1) \leq U$.

This is essentially a 2,3-wheel sieve of Eratosthenes. The asymptotic complexity is unchanged from the classical sieve, $O(N \log \log N)$, but the constant factor improves: only one third of integers are candidates, and no work is spent marking multiples of 2 or 3. On Apple Silicon's unified memory architecture the sieve admits a memory-bandwidth-efficient scatter-write implementation; an MLX-based segmented version capable of ranges up to $\sim 10^{10}$ on a 192 GB M-Ultra accompanies this paper.

Security note. The $6n\pm1$ form is not a restriction on the prime space: every prime > 3 has this form regardless of how it was generated. RSA's standard prime-selection routines (random odd integer, trial division by small primes, Miller–Rabin) implicitly produce $6n\pm1$ primes since no other primes exist above 3. The Furui sieve is mathematically transparent — the joint distribution of (p, q) it produces is statistically identical to that of any sound prime generator at the same bit length and rejection threshold. It therefore contributes nothing to, and detracts nothing from, the cryptographic security of the resulting RSA instance.

2.3 ML-KEM-768

ML-KEM (FIPS 203) is the NIST-standardized form of the CRYSTALS-Kyber lattice-based KEM. The 768-parameter instantiation targets NIST Category 3 security ($\approx$ 192-bit classical, $\approx$ 192-bit quantum against generic attacks). The scheme has three algorithms: KeyGen produces a 1184-byte public key and 2400-byte secret key; Encaps takes a public key and outputs a 1088-byte ciphertext together with a 32-byte shared secret; Decaps takes a secret key and ciphertext and reproduces the 32-byte shared secret deterministically (or, on invalid input, returns an implicitly rejected pseudorandom value).

Security rests on the Module Learning With Errors (M-LWE) problem: distinguishing samples $(A, b = As + e)$ from uniform, where A is a random matrix over $R_n = \mathbb{Z}\_q[X]/(X^{256} + 1)$, s is a small secret vector, and e is a small error vector. No subexponential quantum or classical algorithm against M-LWE is known; the best known attacks are lattice-reduction-based (BKZ with sieving) and run in exponential time.

2.4 Hybrid composition

The standard hybrid KEM construction, formalized in [5, 6], runs both component KEMs in parallel and combines their shared secrets with a key-derivation function:

$$ss_S = KDF(ss^{RSA} \parallel ss^{NLNHM}, transcript)$$

The combiner has the property that ss_S is indistinguishable from random as long as at least one of (ss^{RSA}, ss^{NLNHM}) is indistinguishable from random to the adversary. Formally this is the dual-PRF assumption on the KDF: it acts as a PRF whether keyed by its first or second input, with the other input acting as an

arbitrary message. Modern KDFs based on HMAC-SHA-256 or HMAC-SHA-3 satisfy this assumption [7]. The TLS 1.3 hybrid key-exchange draft [8] uses precisely this construction.

3. The Hybrid KEM

3.1 Notation

Let RSA = (RSA.KeyGen, RSA.Encaps, RSA.Decaps) denote RSA-KEM-2048 in the form of RFC 5990 [9], and let MLKEM = (MLKEM.KeyGen, MLKEM.Encaps, MLKEM.Decaps) denote ML-KEM-768. Let H be SHA-256 and let HKDF be HKDF-SHA-256 [10]. Concatenation is denoted ‖.

3.2 Key generation

```
HybridKEM.KeyGen():
   (pk_R, sk_R) <- RSA.KeyGen(2048)        # uses Furui sieve internally (sec. 4)
   (pk_M, sk_M) <- MLKEM.KeyGen()
   pk = pk_R || pk_M
   sk = sk_R || sk_M || pk_R || pk_M       # pk components retained for transcript
   return (pk, sk)
```

Public-key wire size: 256 + 1184 = 1440 bytes (plus length tags). Secret-key storage: 256 (RSA n + e) + 2400 (ML-KEM sk) + 1440 (cached pk) = 4096 bytes. Implementations that recompute pk from sk on demand can shrink storage to 2656 bytes.

3.3 Encapsulation

```
HybridKEM.Encaps(pk = pk_R || pk_M):
   (ct_R, ss_R) <- RSA.Encaps(pk_R)        # 256-byte ct, 32-byte ss
   (ct_M, ss_M) <- MLKEM.Encaps(pk_M)      # 1088-byte ct, 32-byte ss
   ct = ct_R || ct_M                       # 1344 bytes total
   ss = Combine(ss_R, ss_M, ct_R, ct_M, pk_R, pk_M)
   return (ct, ss)
```

3.4 Decapsulation

```
HybridKEM.Decaps(sk, ct = ct_R || ct_M):
   (sk_R, sk_M, pk_R, pk_M) = parse(sk)
   ss_R <- RSA.Decaps(sk_R, ct_R)
   ss_M <- MLKEM.Decaps(sk_M, ct_M)        # implicit rejection inside
   ss = Combine(ss_R, ss_M, ct_R, ct_M, pk_R, pk_M)
   return ss
```

3.5 KDF combiner

We use HKDF-SHA-256 in extract-then-expand form. The salt is a fixed domain-separation tag; the IKM is the concatenation of both component shared secrets; the info string is the full transcript of ciphertexts and public keys, binding the output to this specific session.

```
Combine(ss_R, ss_M, ct_R, ct_M, pk_R, pk_M):
   PRK = HKDF-Extract(salt = "Hybrid-RSA2048-MLKEM768-v1",
                      IKM  = ss_R || ss_M)
   ss  = HKDF-Expand(PRK,
                     info = ct_R || ct_M || pk_R || pk_M,
                     L    = 32)
   return ss
```

Properties: (i) the fixed salt provides domain separation between this construction and any other use of HKDF-SHA-256 in the same deployment; (ii) the info string binds both ciphertexts and both public keys,

preventing unknown-key-share and re-keying attacks; (iii) the dual-PRF assumption on HMAC-SHA-256 [7] gives ss ≈ uniform if either ss^R or ss^M is uniform; (iv) the 32-byte output is suitable for direct use as an AEAD key (AES-256-GCM, ChaCha20-Poly1305).

Salt versioning. The salt incorporates a version tag ("v1") so a future revision of the hybrid (e.g., RSA-2048 + ML-KEM-1024, or a different KDF) produces shared secrets that cannot collide with this version's output, even if an adversary controls the input to one component. A fixed (rather than per-session) salt is safe here because the info parameter already binds the per-session transcript; the salt's role is purely domain separation across versions and across uses of HKDF in the same deployment.

3.6 Failure handling

ML-KEM-768 uses implicit rejection: an invalid ciphertext does not raise an error but yields a deterministic-but-pseudorandom shared secret derived from the secret key and the ciphertext. RSA-KEM in our construction follows the analogous pattern: an out-of-range or malformed RSA ciphertext is processed in constant time and yields a pseudorandom 32-byte secret derived from sk_R and ct_R. Both branches of decapsulation always run; neither branch is short-circuited.

Concretely, HybridKEM.Decaps satisfies three properties that close the principal oracle channels:

- **Constant-time both branches.** RSA.Decaps and MLKEM.Decaps both run regardless of whether either input parses; their running times must not depend on whether ct_R or ct_M is well-formed.
- **Combiner runs unconditionally.** The combiner Combine(...) is invoked with whatever ss^R and ss^M each component returned (a real shared secret on success, a pseudorandom rejection value on failure), and the 32-byte output is returned without further checks. The caller cannot distinguish a hybrid-success from a hybrid-failure ciphertext from the decapsulation output alone.
- **No partial-success oracle.** An adversary who supplies a valid ML-KEM ciphertext together with a malformed RSA ciphertext receives a 32-byte pseudorandom value bound to the malformed RSA input; an adversary who supplies a valid RSA ciphertext together with a malformed ML-KEM ciphertext receives a 32-byte pseudorandom value bound to the malformed ML-KEM input. In neither case does the adversary learn which component failed, nor any partial information about the surviving secret.

Together these properties ensure that the hybrid inherits IND-CCA security from each component without introducing a new chosen-ciphertext oracle. Implementations should subject the failure paths to the same constant-time scrutiny as the success paths.

4. Furui-Sieved RSA Prime Generation

4.1 Algorithm

RSA.KeyGen at 2048 bits requires two ~1024-bit primes p, q with $\gcd(e, (p-1)(q-1)) = 1$ and $|p - q|$ sufficiently large. Standard implementations alternate between (a) random odd-integer sampling, (b) trial division by small primes, and (c) Miller–Rabin probabilistic primality testing. Step (b) is the dominant cost in fast RSA key generation.

The Furui 6n±1 sieve is a structured replacement for steps (a) and (b) when applied to a windowed range of candidates near a target magnitude. Because the sieve produces only candidates of the form 6n±1 and

removes those with a prime factor ≤ √(window_max), it eliminates the majority of composites in the candidate window before Miller–Rabin runs.

```
FastRSAPrime(target_bits, e):
  while True:
    # Pick a random offset in [2^(b-1), 2^b)
    offset = random_uniform(1 << (target_bits - 1), 1 << target_bits)
    # Sieve a window of ~2^30 candidates around the offset
    window_lo = offset
    window_hi = offset + (1 << 30)
    primes = FuruiSieve(window_lo // 6, window_hi // 6,
                        segment_size = autotune())   # see primes_mlx.py
    for p in primes:
      if not (target_bits - 1 <= p.bit_length() <= target_bits):
        continue
      if not MillerRabin(p, k = 64):
        continue
      if math.gcd(p - 1, e) != 1:
        continue
      return p
    # No usable prime in this window; pick a fresh offset
```

On an Apple M2 Pro with 16 GB unified memory, a 2^{30}-candidate window sieves in roughly 100 ms; the resulting candidate list yields tens of thousands of probable primes per call. Combined with batched Miller–Rabin on the GPU, RSA-2048 keypair generation drops to roughly 150 ms wall time.

4.2 Statistical correctness of the sieve output

We need the output of FastRSAPrime to be indistinguishable from primes produced by a "reference" generator (e.g., random odd integer + trial division + Miller–Rabin) at the same bit length. We give two arguments: a formal one based on rejection sampling, and an empirical one (Appendix B).

Rejection-sampling argument. Both the Furui sieve and the standard reference generator can be viewed as rejection samplers over the same underlying population — the primes in [2^(b−1), 2^b]. The Furui sieve enumerates the candidates of form 6n±1 inside a uniformly-chosen window, applies a wheel-2,3 filter (mark all (6a±1)(6b±1) ≤ window_max), and then returns the survivors that additionally pass Miller–Rabin with k=64. The reference generator samples uniformly random odd integers in the same range, applies trial division up to a fixed bound, and then returns Miller–Rabin survivors. Both filters are conservative (they never reject a prime) and at the cryptographic bit-length Miller–Rabin's false-positive rate is $< 2^{-128}$. The conditional distribution of the output given a uniform window is therefore the uniform distribution over primes in that window, regardless of which filter was used. Composing with a uniform choice of window yields a generator whose output distribution is the uniform distribution over primes in [2^(b−1), 2^b], identical for both methods.

Empirical validation. Appendix B reports Dirichlet-uniformity tests on the residue classes p mod q for q ∈ {5, 7, …, 97} and a prime-number-theorem density agreement test, both run on 375,211 primes produced by the sieve. All chi-square goodness-of-fit p-values exceed 0.95; the observed density agrees with the prime-counting estimate within 7%. Neither test detects any bias against the null hypothesis that the sieve's output is statistically indistinguishable from the population of primes in the window.

Performance, not security. The sieve changes how primes are found, not what is found. The resulting modulus N = pq inherits the standard RSA-2048 hardness assumption with no special-form attack

opened: in particular, no Coppersmith-with-extra-structure variant, Williams's p+1, or Pollard's p−1 attack is enabled by the choice of generator.

5. Security Analysis

5.1 Adversary models

We consider three adversary classes:

- A_classical: classical PPT, with adaptive oracle access to encapsulation and decapsulation queries.
- A_quantum-pre-CRQC: classical for cryptanalytic operations; may run quantum auxiliary computations but cannot run a CRQC against the protocol's lifetime keys. Functionally equivalent to A_classical for our purposes.
- A_quantum-post-CRQC: full quantum PPT, capable of running Shor's algorithm to factor any RSA modulus encountered.

5.2 Hybrid IND-CCA security

Theorem 1 *(Hybrid IND-CCA, informal statement; see Appendix A for the full reduction with advantage bounds). For any (t, q)-bounded adversary A against IND-CCA of HybridKEM (t = total runtime, q = decapsulation queries), there exist:*

- an IND-CCA adversary B_R against RSA-KEM-2048 with runtime t_R = t + O(q · (T_MLKEM + T_HKDF));
- an IND-CCA adversary B_M against ML-KEM-768 with runtime t_M = t + O(q · (T_RSA + T_HKDF));
- a dual-PRF distinguisher D against HKDF-SHA-256 with runtime t_D ≤ t + O(q),

such that:

$$Adv_{HybridKEM}^{IND\text{-}CCA}(A) \le \min(Adv_{RSA}^{IND\text{-}CCA}(B_R),\ Adv_{MLKEM}^{IND\text{-}CCA}(B_M)) + 2 \cdot Adv_{HKDF}^{dual\text{-}PRF}(D)$$

where T_RSA, T_MLKEM, T_HKDF are the per-call runtimes of the components.

Proof sketch. Either ss^R is indistinguishable from uniform (RSA-KEM is secure against B_R) or ss^M is (ML-KEM is secure against B_M). The dual-PRF assumption on HKDF then upgrades that one-sided uniformity into uniformity of the combined output ss = HKDF($ss^R \| ss^M$, info). A standard hybrid argument across the two cases yields the bound. The factor 2 on the dual-PRF advantage absorbs the case split (unknown which component is the secure one). Decapsulation queries are simulated by the reduction in the natural way, using implicit rejection (§3.6) on the failing component. The full reduction is given in Appendix A.

Concrete consequences for our adversary classes:

- **Against A_classical (today):** both Adv_RSA and Adv_MLKEM are conjecturally negligible, so Adv_HybridKEM is at most twice the dual-PRF advantage of HKDF-SHA-256 — itself negligible under standard assumptions [7].
- **Against A_quantum-post-CRQC:** Adv_RSA(B_R) becomes 1 (Shor recovers the factorization, hence the secret key, hence ss^R for any ciphertext). The min(·) collapses to Adv_MLKEM(B_M),

which remains negligible under M-LWE. The hybrid degrades gracefully to pure ML-KEM-768 security.

5.3 The Furui sieve in the security argument

The Furui sieve does not enter the security proof. It is a prime-generation primitive whose output is statistically identical to any sound generator at the same parameters. Therefore the RSA assumption applies to the resulting modulus N = pq with the same parameters as any other RSA-2048 instance. In particular: there is no “FuruI-attack”, and no special-form attack (Coppersmith with extra structure, Williams's p+1, Pollard's p−1, or related) is enabled by the sieve.

5.4 Side-channel mitigations

We address the side-channel surface in three layers: the RSA decapsulation path, the ML-KEM-768 decapsulation path, and the Furui-sieve / GPU Miller–Rabin code path used during key generation.

RSA decapsulation. The standard mitigations apply: constant-time Montgomery multiplication, multiplicative blinding ($m \rightarrow m \cdot r^e$ before exponentiation, then divide r out after), exponent blinding ($d \rightarrow d + k \cdot \varphi(N)$ for fresh k), and CRT recombination performed in constant time with respect to whether p > q. Production libraries (BoringSSL, libsodium, Apple CommonCrypto) implement all of these.

ML-KEM-768 decapsulation. Implicit rejection must execute the rejection branch in constant time and access the same memory pattern as the success branch. Reference implementations (PQClean, liboqs, mlkem-native) do this correctly when compiled at -O2 or -O3 without aggressive branch-on-secret optimisations. Verify with constant-time test vectors and dudect-style timing measurements before production deployment.

Furui sieve and GPU Miller–Rabin. Most of the sieve's data is public — the candidate set 6n±1 in [2^(b−1), 2^b], the small-factor product table, and the surviving candidate list — so these arrays do not require constant-time treatment. The secret data is the random offset within [2^(b−1), 2^b] used to seed the search window, plus the chosen prime p once it is identified. The following mitigations apply:

- Random-offset selection: draw the offset from a CSPRNG (AES-CTR-DRBG or ChaCha20Rand) in fixed-size words; do not branch on its bits.
- Sieve buffer and candidate list: zeroize after use via a memory-barrier-guarded explicit_bzero call (libsodium sodium_memzero) so the buffer is not retained in unified memory after key generation completes.
- Miller–Rabin witnesses: derive each round's witness deterministically from a fixed sequence (e.g., the first k primes 2, 3, 5, …, 311 for k = 64), not from the candidate p. This makes the Miller–Rabin sequence input-independent and removes one secret-dependent branch class.
- Constant-time modular exponentiation in MR: use Montgomery ladder or windowed sliding-window with table accesses on every scan position. The MR test on the GPU should mirror the CPU implementation here.
- Batch shape independence: when the GPU batches Miller–Rabin tests across multiple candidates, the batch shape must be fixed (always k rounds × n candidates) so a small batch's runtime does not leak which candidates were eliminated early. We pad to a fixed batch size.

- Final selection: when one candidate passes all rounds and is returned, the loop should be unrolled to a fixed iteration count so an external observer cannot infer the index of the winning candidate from the Miller–Rabin batch's wall time.
- Memory-zeroization on exit: the chosen prime p remains a secret; the candidate list and intermediate values must be zeroized as above.

We do not claim the supplied reference code (primes_furui.py, primes_numba.py, primes_mlx.py) is constant-time as written; it is intended for performance demonstration. A production implementation should integrate the mitigations above and pass dudect or ctgrind verification before being used to generate cryptographic keys.

5.5 Other caveats

- **Active downgrade.** A network adversary cannot strip the PQC component without breaking the transcript binding in the info parameter, but applications must reject malformed or unexpected ciphertext lengths before invoking decapsulation. Specifically, parsers must check that ct = ct_R || ct_M has the exact expected total length, that ct_R decodes as a valid RSA-2048 ciphertext (length 256 with leading byte in range), and that ct_M decodes as a valid ML-KEM-768 ciphertext (exact length 1088). Any deviation should raise an early protocol error, not be silently truncated.
- **Long-term key reuse.** RSA keys generated today may be exposed retroactively if a CRQC arrives during their lifetime. Hybrid mode protects forward-secret session keys, not the long-term RSA private key. See §7.5 for rotation guidance.
- **No new composition attack.** The dual-PRF combiner introduces no attack surface beyond the two components and the HKDF assumption — no “hybrid attack” exists that does not reduce to breaking a component or the dual-PRF property of HKDF.

6. Implementation

6.1 Apple Silicon target

The primary deployment target is macOS / iOS on Apple Silicon (M1–M5). Implementation choices:

- **RSA-2048:** BoringSSL or libsodium primitives compiled for arm64 with NEON. Prime generation replaced by a wrapper around the Furui sieve described in §4. The sieve itself is implemented in MLX for the wheel pass and in Numba parallel CPU for windowed Miller–Rabin.
- **ML-KEM-768:** PQClean or liboqs reference implementation, compiled with -march=armv8-a+crypto for AES intrinsics. SHA-3/SHAKE acceleration via the M-series CPU implementation; ML-KEM is CPU- and bandwidth-bound, so GPU offload is not currently a win.
- **HKDF:** CommonCrypto kCCHmacAlgSHA256 or libsodium crypto_kdf_hkdf_sha256_*.

6.2 Wire format

For TLS 1.3 hybrid key exchange, the construction maps to a combined named-group entry in draft-ietf-tls-hybrid-design [8]. We adopt the convention that the RSA-2048 public exponent is fixed at e = 65537 and not transmitted on the wire; only the 2048-bit modulus n is sent. This matches existing TLS 1.3 RSA conventions and gives the cleanest byte counts:

Wire field	Bytes

RSA-2048 modulus n (big-endian, fixed length)	256
ML-KEM-768 public key (FIPS 203 §7.1)	1184
Total ClientHello key share	1440
RSA-2048 ciphertext (big-endian, fixed length)	256
ML-KEM-768 ciphertext (FIPS 203 §7.2)	1088
Total ServerHello key share	1344

Encoding rules. Both components are fixed-length on the wire — no length prefixes internal to the named-group payload — so parsing reduces to splitting at the documented offsets (256 for ClientHello, 256 for ServerHello). Outer length fields follow the TLS extension framing of [RFC 8446 §4.2.8]. Endpoints MUST verify the received key_share extension length is exactly 1440 (ClientHello) or 1344 (ServerHello) before invoking decapsulation; any other length is a protocol error and MUST abort the handshake. This length-check is the primary defence against negotiation-stripping or downgrade-to-classical-only attacks (§5.5).

Implementations that need to support variable RSA exponents may add a length-prefixed exponent field (typically 4–8 bytes) and update the totals accordingly; the security argument is unchanged.

For HPKE [13] envelope encryption, the construction fits as a custom KEM ID (in the experimental range 0xFE00–0xFEFF) with the same component layout. ClientHello bloat relative to a pure X25519 + ML-KEM-768 deployment is ~256 bytes from the RSA component; this is acceptable for the use cases where a hybrid with classical RSA (rather than X25519) is mandated, e.g., FIPS 140-3 boundaries or RSA-anchored authentication.

6.3 Performance estimates

Single-thread approximate timings on Apple M2 Pro, representative of mid-range Apple Silicon (microseconds unless noted):

Operation	Time
Furui-sieve RSA-2048 KeyGen (this work)	~150 ms
OpenSSL RSA-2048 KeyGen (reference)	~400 ms
ML-KEM-768 KeyGen	~50 µs
Hybrid KeyGen (dominated by RSA)	~150 ms
RSA-2048 Encaps (public-key op)	~100 µs
ML-KEM-768 Encaps	~50 µs
Hybrid Encaps	~160 µs
RSA-2048 Decaps (private-key op, with CRT)	~3 ms
ML-KEM-768 Decaps	~60 µs
Hybrid Decaps	~3.1 ms

Hybrid latency is dominated by RSA's private-key operation; ML-KEM-768 is two orders of magnitude faster for both operations. Standard RSA optimizations (CRT decryption, blinding, Montgomery multiplication) apply unchanged.

Methodology. These figures are calibrated to public benchmarks for the same primitives on M2 Pro hardware and are intended as engineering estimates, not precise measurements. The full reproducibility protocol — hardware identification (chip revision, RAM, OS version), compiler version and flags, RNG

source, warmup, thermal-monitoring exclusions, and reporting of median plus 5th and 95th percentiles over ≥10,000 runs — is given in Appendix C, together with the code path that produced each timing.

7. Limitations and Discussion

7.1 Hybrid is a transition, not a permanent design

When CRQCs reach maturity, RSA's contribution to the hybrid drops to zero security and pure overhead (in latency, bandwidth, and key-management complexity). The natural successor design is pure ML-KEM-768 — or ML-KEM-1024 for a higher security level — possibly with an X25519 hedge during the early-CRQC period in case unknown lattice attacks emerge. The construction in this paper is engineered for the pre-CRQC and early-CRQC window, not for the post-CRQC steady state.

7.2 ML-KEM is comparatively young

Module-LWE has been studied since 2010 and ML-KEM-768 specifically since 2017, which is short compared to RSA's four decades. Hybrid mode provides exactly the insurance the industry currently desires: if an unforeseen classical attack on M-LWE emerges, RSA still protects pre-CRQC traffic. The economic rationale for hybrid does not, however, scale beyond the early-deployment period; once ML-KEM-768 has accumulated the same decades of cryptanalytic scrutiny RSA enjoys, hybrid becomes pure cost.

7.3 The Furui sieve is orthogonal

We emphasize again that the Furui sieve is a prime-generation primitive, not a cryptographic primitive. A reasonable design treats it as a drop-in optimization for RSA's existing prime selection. We include it in this paper for two reasons: (a) the construction is naturally Apple-Silicon-targeted, and the sieve's GPU implementation accelerates key generation meaningfully on that platform; (b) it provides a tangible answer to the question “can the 6n±1 sieve be used for post-quantum cryptography” by showing exactly where it fits (prime generation for the classical component) and where it does not (the security-providing component, which must remain a quantum-resistant primitive).

7.4 Key rotation policy

RSA-2048 is acceptable as the classical component of this hybrid only for the transition window — roughly until a CRQC capable of factoring 2048-bit moduli becomes operational. We give the following concrete rotation guidance, calibrated to the Gidney–Ekerå estimates [2] and the NSA CNSA 2.0 timeline [11]:

- **Ephemeral RSA keys (per-session in TLS/HPKE):** generated fresh per handshake; no rotation needed beyond the session lifetime. The Furui sieve makes this cheap (§6.3).
- **Long-term RSA-2048 identity keys (e.g., for RSA-anchored authentication):** rotate at most annually, preferably semi-annually, until 2030. After 2030, treat any RSA-2048 key whose corresponding ciphertexts may have been recorded as compromised and migrate the associated identity to ML-DSA (FIPS 204 [14]) or SLH-DSA (FIPS 205 [15]).
- **RSA-2048 retirement target:** 2032–2035 for confidentiality (consistent with CNSA 2.0 [11]), earlier if intelligence-grade CRQC estimates accelerate. Continue accepting RSA-2048 in the hybrid only while ML-KEM-768 alone is considered insufficiently mature.

- **Long-lived signed artifacts (firmware, code-signing, long-retention archives):** do not use RSA-PSS today for signatures intended to be valid past 2030. Use SLH-DSA or ML-DSA, possibly in a hybrid with Ed25519 for the early-deployment hedge.

These dates are advisory and reflect the conservative end of currently published estimates. Operators should track the NIST PQC migration timeline and tighten the schedule if cryptanalytic results warrant.

7.5 Migration recommendations

For new deployments today:

1. Use this hybrid scheme (or the more common X25519 + ML-KEM-768 variant) for ephemeral key exchange in TLS 1.3, IKEv2, SSH, and HPKE.
2. Continue using classical signatures (RSA-PSS, Ed25519) for authentication during the transition, with a planned migration to ML-DSA (FIPS 204) [14] or SLH-DSA (FIPS 205) [15] over 2025–2028.
3. Plan for retirement of classical components from confidentiality once CRQC capabilities approach operational thresholds.
4. For long-term key escrow or signed artifacts that must survive CRQC arrival, prefer SLH-DSA or ML-DSA today over RSA-PSS.

8. Conclusion

We have presented a concrete hybrid KEM combining RSA-2048 (with Furui-sieved prime generation) and ML-KEM-768. The construction is straightforward concatenation under a dual-PRF KDF combiner, and inherits IND-CCA security from either component. The Furui sieve plays no security role but offers an efficiency benefit on Apple Silicon during the transition window. We hope this reference model helps practitioners reason about hybrid PQC deployments and provides a template for both implementation and security analysis.

Appendix A. Formal IND-CCA Reduction

We give the full statement of Theorem 1 and a reduction. Let RSA-KEM = (KG_e, E_e, D_e) and ML-KEM-768 = (KG^M, E^M, D^M) be IND-CCA secure KEMs with shared-secret length 32 bytes each. Let HKDF be the extract-then-expand construction of [10] instantiated with HMAC-SHA-256.

Theorem 1 (formal). *For any (t, q)-bounded IND-CCA adversary A against HybridKEM (t total runtime, q decapsulation queries),*

$$Adv_{HybridKEM}^{IND\text{-}CCA}(A) \le \min\{ Adv_{RSA}^{IND\text{-}CCA}(B_R),\ Adv_{MLKEM}^{IND\text{-}CCA}(B_M) \} + 2 \cdot Adv_{HKDF}^{dPRF}(D)$$

where B_R, B_M, D are explicit reductions running in time at most t + O(q · (T_RSA + T_MLKEM + T_HKDF)) and making at most q oracle queries each. T_X denotes the per-call cost of primitive X.

A.1 Game hops

We define a sequence of games G_0, G_1, G_2, G_3 where G_0 is the IND-CCA experiment for HybridKEM and G_3 has the challenge shared secret replaced by uniform randomness independent of the challenge ciphertext.

$G_0 \rightarrow G_1$: Replace ss^R in the challenge encapsulation with uniformly random $s\tilde{s}^R \in \{0,1\}^{256}$. By IND-CCA security of RSA-KEM the distinguishing advantage of any (t, q)-bounded adversary between G_0 and G_1 is at most Adv_RSA^IND-CCA(B_R), where B_R simulates the ML-KEM and HKDF components and forwards RSA decapsulation queries to its own oracle.

$G_1 \rightarrow G_2$: Replace the challenge HKDF output $ss = HKDF(s\tilde{s}^R \| ss^M, info)$ with HKDF keyed by uniform randomness independent of ss^M. By the dual-PRF property of HKDF (with key $s\tilde{s}^R$ and message $ss^M \|$ info), the distinguishing advantage is at most Adv_HKDF^dPRF(D_1).

$G_2 \rightarrow G_3$: Symmetrically, the alternate hop $G_0 \rightarrow G_1'$ replacing ss^M first costs at most Adv_MLKEM^IND-CCA(B_M); the subsequent hop costs at most Adv_HKDF^dPRF(D_2). The adversary must succeed against either path, so the bound takes the minimum of the two component-IND-CCA advantages and adds the dual-PRF advantage twice (once per path).

In G_3 the challenge shared secret is uniform randomness independent of the challenge ciphertext, so the adversary's advantage in G_3 is exactly 0. Composing the bounds across hops yields the inequality of Theorem 1.

A.2 Decapsulation simulation

Each reduction must answer A's q decapsulation queries. B_R simulates ML-KEM and HKDF locally (it knows sk^M from the simulation setup), and forwards the RSA portion of each query to its own RSA-KEM decapsulation oracle. B_M does the symmetric thing. The implicit-rejection behavior of both components (§3.6) ensures that decapsulation of any malformed query yields a deterministic-pseudorandom value that the simulator can compute without knowledge of either secret key, preventing oracle leakage. D's simulation is straightforward: it answers all queries by invoking HKDF (in either keying direction) on the simulated component secrets.

A.3 Tightness

The reduction is tight up to the factor of 2 on the dual-PRF advantage, which is unavoidable for this combiner template (the adversary may exploit either HKDF keying direction). The dependence on the runtime overhead T_RSA + T_MLKEM + T_HKDF is constant per query, so for typical deployments where q is small relative to t, the runtime of the reductions is dominated by t itself. For the concrete numerical bounds at RSA-2048 and ML-KEM-768 parameters, see [4, 5, 6].

Appendix B. Empirical Validation of the Furui-Sieve Distribution

We empirically verify the rejection-sampling argument of §4.2. Two tests are reported here; both are reproducible from the supplementary code repository (primes_furui.py).

B.1 Experimental setup

We sieve all primes in the window W = [6,000,005, 12,000,001] using the vectorized NumPy implementation of the Furui sieve (primes_furui.py, find_primes_furui(x=1,000,001, y=2,000,000)). The output is the exact set of all primes greater than 3 in W, totalling 375,211 primes. We do not sub-sample; the test population is the full prime set. This range is chosen to be small enough for exhaustive enumeration in seconds yet large enough for stable chi-square statistics; the rejection-sampling argument extends the conclusion to RSA-magnitude primes.

B.2 Test 1 — Dirichlet uniformity of p mod q

By Dirichlet's theorem, the residue classes (p mod q) for primes p > q are equidistributed across the φ(q) classes coprime to q. For prime q this means uniformity over {1, …, q−1}. We test this for a range of small primes q using a Pearson chi-square goodness-of-fit test against the uniform distribution; p-values are computed via the Wilson–Hilferty normal approximation (accurate to 3 decimal places for the degrees of freedom we report).

q	χ^2	dof	p-value	Result (α = 0.05)
5	0.212	3	0.9701	pass
7	0.478	5	0.9910	pass
11	0.496	9	0.9999	pass
13	1.303	11	0.9997	pass
17	1.201	15	1.0000	pass
19	2.243	17	1.0000	pass
23	2.177	21	1.0000	pass
29	3.358	27	1.0000	pass
31	3.657	29	1.0000	pass
37	8.173	35	1.0000	pass
41	4.780	39	1.0000	pass
43	7.255	41	1.0000	pass
47	7.307	45	1.0000	pass
53	6.556	51	1.0000	pass
97	19.775	95	1.0000	pass

All 15 chi-square statistics are well below the α = 0.05 critical value (which ranges from 7.815 at dof = 3 to 118.752 at dof = 95). The high p-values (most ≥ 0.99) reflect that we are testing the complete population of primes in the window — the natural variance of a true sample is reduced when the population itself is exhaustively enumerated. No bias against uniform distribution is detected.

B.3 Test 2 — PNT density agreement

The prime-number theorem predicts the count of primes in [a, b] to be approximately b/ln(b) − a/ln(a). For W we obtain:

Quantity	Value
PNT estimate b/ln b − a/ln a	351,741
Sieve-observed count	375,211
Ratio observed / PNT	1.067
Result	pass (ratio ≈ 1 ± O(1/ln a) for windows of this size)

The 6.7% over-count is the expected residual term in the prime-counting approximation π(x) ≈ x/ln(x); the more precise Li(x) approximation agrees within a fraction of a percent. The sieve enumerates the full prime population without omission.

B.4 Conclusion

Both tests are consistent with the rejection-sampling argument of §4.2: the Furui sieve produces the complete and unbiased set of primes within its window. Combined with a uniform random window selection, the resulting prime distribution at RSA-magnitude bit lengths matches that of any sound generator. We recommend an extended replication study at 1024-bit prime size as part of any production deployment, comparing against the same reference generator the deployment intends to replace.

Appendix C. Reproducible Benchmark Methodology

The performance figures in §6.3 are calibrated estimates intended for deployment planning. This appendix specifies the protocol an implementer should follow to reproduce or supersede those figures on their own hardware.

C.1 Hardware identification

Report exactly: chip name as returned by sysctl machdep.cpu.brand_string; CPU performance- and efficiency-core counts; total RAM as returned by sysctl hw.memsize; GPU core count from system_profiler SPDisplaysDataType; macOS version (sw_vers); thermal state (pmset -g thermlog) at start and end of run.

C.2 Build configuration

Compiler: clang as bundled with the Xcode version reported. Flags: -O3 -march=armv8-a+crypto for production, with -fstack-protector-strong and -D_FORTIFY_SOURCE=2 for defensive builds. ML-KEM-768: use the PQClean reference implementation at the commit hash reported. RSA: use BoringSSL or libsodium at the commit hash reported, with CRT and blinding enabled. HKDF: CommonCrypto kCCHmacAlgSHA256.

C.3 Randomness

All keypair-generation runs draw from /dev/urandom for seeds, stretched via AES-CTR-DRBG (NIST SP 800-90A) with at least 256 bits of entropy. Time spent in DRBG generation is included in the keygen timing. Runs are not pre-seeded with deterministic test vectors except for interoperability cross-checks (which are reported separately).

C.4 Measurement protocol

- **Warmup.** Discard the first 1,000 operations of each kind to allow JIT compilation, MLX kernel caching, and cache warm-up to settle.
- **Sample size.** ≥ 10,000 operations per measurement, with the system otherwise idle (no other user processes; Spotlight indexing paused).
- **Statistics.** Report median, 5th-percentile, 95th-percentile, and standard deviation. Single-thread runs and multi-thread runs are reported separately.
- **Thermal exclusion.** Monitor pmset -g thermlog throughout; discard any measurement after a thermal-throttle event until the device returns to the Nominal state for at least 60 seconds.
- **Allocation included.** Memory allocation (mlx.zeros, mlx.array creation) and finalization (mlx.eval, sodium_memzero) are included in the timed interval, since they are unavoidable in production.

C.5 Reporting

Publish raw timing logs (CSV: operation, run_index, elapsed_ns, thermal_state) alongside any summary statistics. Provide the exact Python (and where applicable C) source compiled for each component, or commit hashes of the upstream libraries. The accompanying benchmark.py script in this paper's repository follows this protocol for the Furui-sieve component; integrators should extend it with the ML-KEM-768 and HKDF measurements.

C.6 Test vectors

For interoperability, implementations should publish: (1) at least 16 complete (pk, sk, ct, ss) tuples generated with documented seeds, covering both success and ML-KEM/RSA-explicit-rejection paths; (2) the salt and version tag used in the HKDF call; (3) the exact byte ordering of the concatenated ct_R ‖ ct_M and ss_R ‖ ss_M strings. Tests should verify byte-exact equality of derived shared secrets across independent implementations.

References

[1] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS), 1994, pp. 124–134.

[2] C. Gidney and M. Ekerå, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits,” Quantum, vol. 5, p. 433, April 2021.

[3] R. Furui, “The formulation of prime numbers,” 2024. Available at Ryoji.info.

[4] National Institute of Standards and Technology, FIPS 203, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” August 2024.

[5] N. Bindel, U. Herath, M. McKague, and D. Stebila, “Transitioning to a quantum-resistant public key infrastructure,” Post-Quantum Cryptography (PQCrypto 2017), Springer LNCS 10346.

[6] F. Giacon, F. Heuer, and B. Poettering, “KEM combiners,” Public-Key Cryptography (PKC 2018), Springer LNCS 10769.

[7] M. Bellare and B. Tackmann, “The multi-user security of authenticated encryption: AES-GCM in TLS 1.3,” CRYPTO 2016, Springer LNCS 9814.

[8] D. Stebila, S. Fluhrer, and S. Gueron, “Hybrid key exchange in TLS 1.3,” IETF Internet-Draft draft-ietf-tls-hybrid-design.

[9] J. Randall, B. Kaliski, J. Brainard, S. Turner, “Use of the RSA-KEM Key Transport Algorithm in the Cryptographic Message Syntax (CMS),” RFC 5990, September 2010.

[10] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, May 2010.

[11] National Security Agency, “Commercial National Security Algorithm Suite 2.0 (CNSA 2.0),” September 2022.

[12] Bundesamt für Sicherheit in der Informationstechnik (BSI), “Technical Guideline TR-02102-1: Cryptographic Mechanisms — Recommendations and Key Lengths,” 2024.

[13] R. Barnes, K. Bhargavan, B. Lipp, C. Wood, “Hybrid Public Key Encryption,” RFC 9180, February 2022.

[14] National Institute of Standards and Technology, FIPS 204, “Module-Lattice-Based Digital Signature Standard,” August 2024.

[15] National Institute of Standards and Technology, FIPS 205, “Stateless Hash-Based Digital Signature Standard,” August 2024.